

Answers:

1. What problem does using bit flags solve in the context of game development?
 - a. Bitflags allow you to serve and manage multiple boolean values from a single integer by using the individual bits. This simplifies code, reduces risk of error, and conserves memory.
2. If you're using a 16-bit integer to store flags, what is the maximum number of individual boolean values you can represent?
 - a. 16, since this is a 16 bit integer, you can store 16 possible boolean values.
3. Explain the advantage of using shift operators for defining bit flags over just writing the raw binary or decimal numbers.
 - a. It's basically about readability and maintainability, as you can clearly see which bit is being set. You can easily add or remove states, and setting a specific one is less error prone.
4. How does using bit flags contribute to reducing bugs and errors in the codebase?
 - a. It allows you to manage a set of boolean states in a single variable. Making sweeping state changes, like setting them all to 0, becomes trivial with bit flags.
5. When would you prefer using individual boolean variables over bitflags?
 - a. If the states aren't tightly related and don't require bulk operations, it may be preferable to use individual booleans. Additionally, debugging can be easier with individual values rather than bit flags.
6. What operator do you use to toggle an individual bit?
 - a. XOR
7. Given 'playerState', and a mask IS_ANGRY, how would you set IS_ANGRY on 'playerState' without changing anything else?
 - a. `playerState |= IS_ANGRY`
8. Given 'playerState', and a mask IS_ANGRY, how would you toggle IS_ANGRY?
 - a. `playerState ^= IS_ANGRY`
9. Given 'playerState', and a mask IS_ANGRY, how would you clear IS_ANGRY?
 - a. `playerState &= ~IS_ANGRY;`
10. Given 'playerState', and a masks IS_ANGRY, IS_TIRED, how would you unset both of those flags at once?
 - a. `playerState &= ~(IS_ANGRY | IS_TIRED)`
11. Given 'playerState', how would you clear all flags?
 - a. `playerState = 0`
12. Given 'playerState' and the flags IS_ANGRY, IS_HUNGRY, IS_TIRED, how would you set all 3 flags at once, without changing anything else?
 - a. `playerState |= (IS_ANGRY | IS_HUNGRY | IS_TIRED)`
13. Given 'playerState' and the flags IS_ANGRY, and IS_TIRED, how would you check that none of them are set?
 - a. `if ((playerState & (IS_ANGRY | IS_TIRED)) == 0) { // do something }`